
DELIVERY TRACKING APP - CLIENT GUIDE

DriverTrack

An offline-first mobile app that lets drivers manage and confirm deliveries on the spot, even with no internet. This guide covers daily use, and how it works in React Native: local queueing, automatic syncing, duplicate protection and conflict handling.

React Native - TypeScript - September 2026

Table of Contents

Part I - Using the App	3
1. What is DriverTrack?	3
2. Getting Started	3
3. A Tour of the Screens	4
3.1 My Deliveries (the Route tab)	4
3.2 Delivery Details	4
3.3 Marking a Delivery as Delivered	4
3.4 Reporting a Failed Delivery	5
3.5 The Sync Queue Tab	5
3.6 The Network Simulator (the gear icon)	5
4. Understanding the Status Badges	5
5. Demo Script: Eight Scenarios in Ten Minutes	6
6. Frequently Asked Questions	7
Part II - How It Works (Technical Overview)	8
7. Technology Stack	8
8. React Native in a Nutshell	8
9. System Architecture	9
10. The Outbox Pattern: How Offline Actually Works	10
11. Sync States and Rules	10
12. Duplicate Protection (Idempotency)	11
13. Conflicts: When the Server Changed First	11
14. Data Model	11

15. Quality Assurance and Testing	12
16. Repository Workflow and Structure	13
17. Assumptions and Notes	14

Part I - Using the App

This first part is written for anyone who wants to install, run and use DriverTrack: product owners, reviewers and drivers. No programming knowledge is needed for Part I. Part II explains how the app is built, for technical readers.

1. What is DriverTrack?

DriverTrack is a mobile application for delivery drivers. Every morning a driver opens the app and sees the deliveries assigned to them for the day. At each stop the driver opens the delivery, calls or navigates to the customer if needed, and then records the outcome: **Delivered** (with a required recipient name, an optional note and an optional photo) or **Failed** (with a required reason and an optional note).

The defining feature of DriverTrack is that it **keeps working when the internet does not**. Delivery drivers constantly move through areas with weak or missing mobile coverage. If a confirmation is recorded while offline, the app saves it safely on the device, clearly labels it as *Waiting to sync*, and sends it automatically as soon as connectivity returns. Nothing the driver records is ever lost, even if the phone is switched off or the app is closed and reopened.

Offline-first

works with no signal

24 / 24

automated tests passing

9 PRs

reviewed & merged work

3 taps

to confirm a delivery

The app also protects against the classic problems of mobile data entry: it never submits the same confirmation twice (every action carries a unique idempotency key), and if someone else changes a delivery on the server before the driver's update arrives, the app detects the conflict and asks the driver how to resolve it instead of silently overwriting data.

2. Getting Started

DriverTrack is built with React Native and runs on Android (and on iOS from a Mac). The steps below assume a Windows PC and an Android device.

Step	Command	What it does
1	<code>npm install</code>	Installs the JavaScript dependencies (once).
2	<code>npm start</code>	Starts Metro, the development server. Keep this terminal window open.
3	<code>npm run android</code>	Builds the app and installs it on a connected device or emulator. The first build takes 3-10 minutes; later runs are much faster.
4	<code>npm test</code>	Runs the 24 automated tests. No device needed.

Table 1 - Installation and run commands

For the device you have two options: an **Android emulator** created with Android Studio (Device Manager, pick a recent Pixel with a recent API level), or a **physical phone** with USB debugging enabled in Developer options. A real phone is the most realistic way to experience the offline behaviour - you can literally walk into a basement and watch the app queue your updates.

Repository: github.com/yousseffninja/deliveryApp (private). The development history is organised as nine pull requests, each one documented and merged in order - see Section 16.

3. A Tour of the Screens

The app has two bottom tabs - **Route** and **Sync Queue** - plus detail and form screens that open on top. A gear icon in the header opens the Network Simulator used for demonstrations.

3.1 My Deliveries (the Route tab)

This is the driver's home screen. At the top, three stat cards show how many deliveries are pending, delivered and failed today. Below that, filter chips (All / Pending / Delivered / Failed) narrow the list, and each delivery card shows the order number, customer, address, cash amount, payment method and the current status badges. Pulling down refreshes the list from the server. If the device is offline, an orange banner explains how many updates are waiting to sync.

Tapping a card opens the delivery's detail screen. If the list cannot be loaded (for example on first launch with no connection), the app shows a clear error state with a **Try again** button - and, when a previous copy exists, offers the cached list instead of an empty screen.

3.2 Delivery Details

The detail screen gathers everything the driver needs at the stop: the customer's name and phone with a one-tap **Call customer** button, the address with an **Open in Maps** button, the amount due (shown in KWD, three decimals) and the payment method, plus any customer notes such as gate codes. The status badges at the top always show both the delivery status and the sync status.

For a pending delivery, two large buttons sit at the bottom: **Mark as Delivered** and **Report Failed Delivery**. If an update is still waiting to sync, those buttons are replaced by a status card offering **Retry now** or **Discard**, so the driver is never confused about what has and has not reached the server.

3.3 Marking a Delivery as Delivered

The confirmation form asks for the **recipient name** (required - it is pre-filled with the customer's name, and it is what protects the company in disputes), an optional note of up to 240 characters, and an optional photo proof picked from the camera or gallery. Confirming is deliberately fast: three taps for a typical delivery.

3.4 Reporting a Failed Delivery

The failure form requires a **reason** chosen from five standard options (customer unavailable, wrong address, refused delivery, payment could not be collected, other) plus an optional note. Standardised reasons keep the back-office reports meaningful.

3.5 The Sync Queue Tab

The Sync Queue is the transparency centre. Stat cards summarise what is waiting, what failed and what synced today. Queued items are listed in three groups: **Needs attention** (failed or conflicting - each with its error message and a Retry / Discard / Resolve button), **Waiting to sync** (with age and attempt count), and **Recently synced**. Buttons for **Sync now** and **Retry all failed** sit at the top. A red badge on the tab icon always shows how many updates are queued.

3.6 The Network Simulator (the gear icon)

Because the app talks to a mocked backend, the simulator screen can reproduce every network situation on demand: **Online** (normal), **Slow 3G** (4-8 second responses with occasional stalls), **Flaky server** (about 45% of requests fail, exercising the automatic retry logic) and **Airplane mode** (everything queues locally). It can also arm a *server-side conflict* for the next submit, apply a server-side change immediately, and reset the mock data. The chosen profile is saved, so you can go offline, close the app completely, reopen it and see the queue survive - exactly like a real outage.

4. Understanding the Status Badges

Every delivery in the app carries two independent pieces of status. The **delivery status** describes the outcome of the delivery itself; the **sync status** describes whether the record of that outcome has reached the server yet.

Delivery status	Meaning
Pending	Still to be delivered. Actions are available.
Delivered	Handed over successfully; recipient name recorded.
Failed	Could not be completed; reason recorded.
Cancelled	Cancelled on the server side (for example by customer support).

Table 2 - Delivery statuses

Sync status	Meaning	What the app does
Synced	Server has confirmed the update.	Nothing - it is done. Shown for a while in Recently synced.
Waiting to sync	Saved on the device; the server has not received it (usually offline).	Sends automatically when connectivity returns; retries in the background with growing delays.
Syncing	Being sent right now.	Shows a brief progress state.

Sync status	Meaning	What the app does
Failed to sync	Tried several times (3 by default) and the server still did not accept it.	Stops automatic retries and asks the driver to tap Retry now or Discard.
Conflict	The server says the delivery changed before this update arrived.	Opens a dialog: Discard my update, or Keep for dispatch review.

Table 3 - Sync statuses and what they mean

Rule of thumb: a driver can always keep working. Any badge other than Synced simply means "the app has not finished talking to the server yet" - the record itself is already safe on the phone.

5. Demo Script: Eight Scenarios in Ten Minutes

This is the fastest way to experience everything the app does. Open the Network Simulator with the gear icon whenever a step says so.

1. **Normal delivery.** Open any pending delivery, tap Mark as Delivered, confirm. Watch the badge go Syncing then Synced.
2. **Offline queueing.** Simulator: choose Airplane mode. Deliver another stop. The confirmation is accepted instantly and shows Waiting to sync; an orange banner appears on the list.
3. **Survives a restart.** Still offline, close the app completely and reopen it. The queued update is still in the Sync Queue - nothing was lost.
4. **Automatic sync.** Simulator: switch back to Online. Within moments the queue drains by itself and the update turns Synced.
5. **Failing server, automatic retry.** Simulator: choose Flaky server. Deliver another stop and watch the Sync Queue: the attempt count grows, then the update succeeds. Roughly one in three deliveries will exhaust the 3 attempts and park as Failed to sync.
6. **Manual retry.** For any Failed to sync item, tap Retry now (or Retry all failed). You can also switch the simulator to Online first to guarantee success.
7. **Slow network.** Simulator: choose Slow 3G. Deliveries still succeed, but you will see the Syncing state last longer, and occasionally a request stalls and the app times out and retries - all by itself.
8. **Conflict.** Simulator: pick a pending delivery and tap Arm conflict. Now deliver it. The server changes the order just before your update lands and answers 409; the app shows the Delivery-changed-on-server dialog where you choose Discard my update or Keep for dispatch review.

Duplicate protection check: while an update is waiting to sync, the delivery's action buttons are replaced by the status card, so the same confirmation cannot be submitted twice from the UI. On the wire, every retry reuses the same unique client_action_id, so even a duplicate network request cannot create a second confirmation - this is proven by automated tests (Part II, Section 15).

6. Frequently Asked Questions

Q: I recorded a delivery with no signal. Is it saved?

A: Yes. It is written to the phone's persistent storage the moment you confirm, before any network is attempted. It will show Waiting to sync until the server accepts it.

Q: I closed the app / the phone died. Is my update gone?

A: No. Queued updates are persisted and reloaded on the next launch. They survive restarts, force-kills and reboots.

Q: When does the app retry by itself?

A: Immediately when connectivity returns, roughly every 20 seconds while the app is open (with a growing delay per failed attempt), and once at startup if updates are queued. After 3 failed attempts it waits for you to tap Retry so failure stays visible.

Q: Can a delivery be confirmed twice by accident?

A: No. The app allows one pending update per delivery, blocks duplicate submissions in the interface, and the server recognises repeated submissions of the same confirmation and answers with the original result.

Q: What is a conflict, in plain words?

A: Someone changed the delivery on the server (for example the customer cancelled through support) before the driver's confirmation arrived. The app refuses to overwrite that change silently and asks what to do instead.

Q: The photo proof - is it required?

A: No, it is optional. If attached, it is uploaded before the confirmation and only once, even across retries.

Part II - How It Works (Technical Overview)

This part explains how DriverTrack is built with React Native, how the offline synchronisation actually works, and where every behaviour lives in the codebase. It is aimed at technical reviewers and future maintainers.

7. Technology Stack

The task allowed React Native or Flutter; DriverTrack is implemented in **React Native 0.87 (bare CLI) with TypeScript**. Each dependency was chosen to keep the core logic testable without a device.

Technology	Role in the app	Why this choice
React Native 0.87 + TypeScript	Cross-platform UI, one JavaScript codebase rendering native views.	Industry standard, fast iteration, strong typing catches bugs at compile time.
React Navigation v7	Bottom tabs (Route / Sync Queue) + native stack screens.	De-facto standard navigation for React Native.
Zustand	App-wide state store.	Tiny API, works outside React components, trivially testable.
AsyncStorage	Persistent key-value storage.	Reliable, synchronous-API async persistence for the delivery cache and the outbox queue.
NetInfo	Device connectivity events.	Official community library; drives the automatic sync on reconnect.
react-native-image-picker	Attaching photo proof.	Mature picker with camera + gallery support.
Jest	Unit testing.	Standard RN test runner; the core logic tests run in plain Node.
MockServer (in-app)	Fake backend with controllable network behaviour.	Reproduces every required scenario deterministically - no infrastructure needed.

Table 4 - Libraries and their roles

8. React Native in a Nutshell

React Native lets one JavaScript codebase drive real native interfaces. The developer writes **components** - small functions that describe the UI as a function of their data - and React Native maps them onto actual native views (Android's View system, iOS's UIKit). The result looks and behaves like a native app, not a web page in a wrapper.

Three mechanisms matter for understanding this project. First, **state**: the UI is a projection of state, so all screens read from a single store (Zustand) and re-render automatically when it changes. Second, **Metro**: the

bundler that serves the JavaScript to the app, with Fast Refresh showing code changes on the device in about a second. Third, **native modules**: features that need platform APIs - storage, network detection, the image picker - come as small native libraries with a JavaScript API; the app code only ever talks to the JavaScript side.

The practical consequence for DriverTrack: all of the business logic - the queue, the retry rules, the conflict handling - lives in plain TypeScript modules with no React and no native dependencies. That is exactly what makes it testable in seconds with Jest, with no emulator in sight.

9. System Architecture

The codebase is layered so that each layer only talks to its neighbour, and the whole sync engine can run - and be tested - without any UI.

Layer	Location	Responsibility
Screens (UI)	src/screens, src/components	Render state via selectors; call store actions on user input. No fetching, no storage access.
App store	src/store/useAppStore.ts	Single source of truth: deliveries, outbox, sync log, simulator. Persists through repositories and delegates all network work to the engine.
Repositories	src/storage	Typed persistence over AsyncStorage: delivery cache, outbox queue, synced log, simulator settings.
Sync engine	src/sync/syncEngine.ts	Drains the outbox in order with retry/backoff, detects conflicts, emits events back to the store.
Connectivity	src/sync/connectivity.ts	Wraps NetInfo into one boolean stream consumed by the store and engine.
Mock API	src/api	MockServer implements the Api interface: latency profiles, injected failures, offline mode, idempotency registry, 409 conflicts.

Table 5 - Layers and their responsibilities

Data always flows one way around the loop: **Screens -> Store -> SyncEngine -> Mock API**, with results coming back as events that the store applies and persists. Because the engine receives its dependencies (the API, the repositories, a connectivity check) from outside, the tests substitute in-memory doubles for all of them - which is why 24 tests covering the whole offline lifecycle run in about three seconds.

Swapping the mock for a real backend means implementing one TypeScript interface (`src/api/types.ts`) over real HTTP and changing one import. No screen, store or engine code changes.

10. The Outbox Pattern: How Offline Actually Works

DriverTrack uses the classic **outbox pattern**, the same approach used by banking and logistics apps that must never lose a transaction.

1. **Record.** The driver confirms a delivery. The app creates a pending action with a fresh unique id (the `client_action_id`) and writes it to the outbox in persistent storage. This happens before any network call - if the app crashed one millisecond later, the action would still be on disk.
2. **Apply locally.** The delivery in the local cache is optimistically updated (status becomes delivered or failed) so the UI reflects the driver's intent immediately, marked Waiting to sync.
3. **Hand off.** The engine is asked to sync. If the device is online it starts immediately; if not, the action simply waits in the outbox.
4. **Send.** When online, the engine takes queued actions oldest-first, marks one as syncing, uploads the photo proof if present (exactly once), and posts the confirmation with its `client_action_id`.
5. **Confirm or retry.** On success the action is removed, the server's authoritative delivery state is stored, and the item appears under Recently synced. On a retryable failure (no network, timeout, server error) the attempt counter grows and the action goes back to waiting with a longer delay each time - 5s, 10s, 20s, capped at 60s.
6. **Park or escalate.** After 3 failed attempts the action is parked as Failed to sync and left for a manual retry, keeping the failure visible instead of hiding it. A 409 conflict is parked separately with the server's state for the resolution dialog.

11. Sync States and Rules

Each queued action moves through a small, explicit state machine.

State	Meaning	Moves on
waiting	Saved, not yet accepted by the server.	Engine picks it up when online (submit, reconnect, 20s timer respecting backoff).
syncing	A request is in flight.	Success -> removed (synced); failure -> waiting / failed / conflict.
failed	Exhausted automatic attempts or hit a permanent error.	Manual Retry resets attempts and re-enters the queue.
conflict	Server returned 409 with its own state.	User decision only: Discard my update, or Keep for dispatch review.
synced	Server accepted it; the action is deleted from the outbox and logged.	Terminal. Appears in Recently synced.

Table 6 - The sync state machine

Automatic triggers are: connectivity returning (NetInfo event), the app starting while online, a 20-second background timer, and the moment a driver submits. Failed actions deliberately require a human tap - the task asks for failures to remain visible and manually retryable.

12. Duplicate Protection (Idempotency)

A slow network can do strange things: a request may reach the server and succeed, but the acknowledgement is lost, so the client retries. Without protection the customer would be marked delivered twice. DriverTrack prevents this with three independent layers:

Layer	Protection
UI	One pending action per delivery; while it exists, the action buttons are replaced by the sync status card.
Client engine	All sync attempts are serialised (one queue drain at a time) and every attempt reuses the same <code>client_action_id</code> .
Server	The mock server keeps a registry of processed <code>client_action_ids</code> . A repeat submission returns the original result marked duplicate and does not change state again.

Table 7 - Three layers of duplicate protection

13. Conflicts: When the Server Changed First

Between the moment a driver taps confirm and the moment the request arrives, the delivery may have changed on the server - the classic case is a customer cancelling through support. The mock server rejects such a submit with HTTP 409 and includes its current state.

The engine parks the action as **conflict** - it is never retried automatically - and the app shows the *Delivery changed on server* dialog. **Discard my update** deletes the queued action and reverts the local view to pending (a later refresh reconciles everything). **Keep for dispatch review** leaves the action parked in the Needs attention section where it can later be discarded or retried. The point is that the app never silently overwrites someone else's change, and never leaves the driver guessing.

14. Data Model

Two structures carry the whole feature. Field names mirror the task's suggested API contract.

Delivery field	Type	Notes
id / order_number	number / string	Server identity (1001, ORD-1001).
customer_name / phone	string	Shown on cards and the details screen; one-tap call.

Delivery field	Type	Notes
address	string	Opens in Google Maps.
amount_due	number	KWD, three decimals.
payment_method	cash card	Rendered as a chip.
status	pending delivered failed cancelled	Server-owned delivery outcome.
recipient_name / note / failure_reason	string null	Filled by the confirmation or failure flow.
version / updated_at	number / timestamp	Bumped on every server mutation; used in conflict responses.

Table 8 - Delivery (server + cache)

PendingAction field	Type	Notes
id = client_action_id	UUID string	Created once on the device; the idempotency key for every attempt.
delivery_id	number	One action per delivery, enforced.
type	complete fail	Which confirmation this is.
status	waiting syncing failed conflict	The state machine of Section 11.
payload	complete or fail payload	Recipient name / reason / note exactly as the API expects.
attempts / last_attempt_at	number / timestamp	Backoff scheduling (5s doubling, 60s cap).
photo_uri / proof_uploaded	local uri / boolean	Proof uploaded once before the complete call.
server_delivery	Delivery null	Attached to conflicts for the resolution dialog.

Table 9 - PendingAction (the outbox entry)

15. Quality Assurance and Testing

The important behaviours are locked down by 24 automated tests that run in plain Node in about three seconds - no emulator required - because the engine, repositories and mock server are plain TypeScript with injected dependencies.

Area	What the tests prove
Mock server (10 tests)	Idempotency: the same client_action_id never applies twice. Conflicts return 409 with server state and are not cached. Offline requests fail and the same action succeeds after reconnect. Photo proof attaches to the delivery.
Persistence (8 tests)	Queued actions and the delivery cache survive an app restart (new repository instances over the same storage). Corrupt storage falls back to safe defaults.

Area	What the tests prove
Sync engine (5 tests)	Happy path, offline no-op, flush on reconnect, retryable 500s with growing attempts, parking as failed after the limit, conflicts never auto-retried, proof uploaded exactly once, backoff maths.
Formatting (1 test)	Money, relative time and reason labels.

Table 10 - Test coverage by area

Command	Purpose
<code>npm test</code>	Runs the 24 unit tests.
<code>npx tsc --noEmit</code>	Strict TypeScript check across the codebase.

Table 11 - Quality commands

16. Repository Workflow and Structure

The repository history mirrors a professional delivery: each feature landed as its own branch, was opened as a documented pull request, and was merged into main in order.

PR	Title
#1	Project scaffold - theme, domain types, tooling
#2	Mock API layer with realistic network behaviour
#3	Local persistence (deliveries cache + durable outbox)
#4	Offline-first sync engine and app state store
#5	Delivery list and details screens with sync status UI
#6	Delivered / failed forms with validation and photo proof
#7	Sync queue, conflict dialog, and network simulator
#8	Core behaviour tests (24 passing)
#9	README, architecture guide, publishing script

Table 12 - The nine merged pull requests

Folder	Contents
<code>src/api</code>	Mock server, network profiles, Api interface, typed errors.
<code>src/storage</code>	KeyValue port + repos for cache, outbox, synced log, simulator.
<code>src/sync</code>	Connectivity wrapper and the SyncEngine.
<code>src/store</code>	Zustand store, selectors, engine wiring.
<code>src/screens, src/components</code>	The six screens and reusable UI (cards, chips, banner, dialogs, states).

Folder	Contents
src/navigation, src/theme	Tabs + stack navigation; design tokens.
tests	The 24 Jest unit tests.
docs	This guide and ARCHITECTURE.md.

Table 13 - Where things live

17. Assumptions and Notes

The task allowed reasonable assumptions; the significant ones are listed here and also appear in the repository README.

- **Conflict definition:** a submit conflicts when the delivery is no longer pending on the server (already delivered, failed or cancelled by someone else). The server response carries the authoritative state.
- **Keep for dispatch review** leaves the conflicting action parked; it never auto-retries and waits for a person to discard or retry it.
- **Failed syncs need a manual retry** after 3 automatic attempts, so failure stays visible, as the task requires.
- **Photo proof** is optional, picked from gallery or camera, stored as a local file and uploaded once before the confirmation.
- **Single driver, no authentication** - the task focuses on the delivery and sync behaviour, not accounts.
- Amounts are in **KWD with three decimals**, matching the sample data.

Next step for production: replace the MockServer with a real HTTP implementation of the same Api interface (one file), keep everything else - the outbox, retries, conflicts and all tests - exactly as they are.